Legality, mapping and residency

Virtual memory separates logical ranges, software mappings, and physical residency. A legal virtual page need not be resident. Present/resident bits have architecture-specific meanings; valid is not one universal cross-architecture name. A handler must separately establish legal range/backing, translation state, and read/write/execute/user permission.

Fix 4 KiB pages and processes A/B with identical virtual layouts.

Virtual range	Legality/rights	Content source	A mapping	B mapping
[0x0000,0x1000)	Legal RX, not W	Same program code	Shared F0	Shared F0
[0x1000,0x2000)	Legal RW private	Zero-fill-on-demand	Nonresident	Nonresident
[0x2000,0x3000)	Legal RW private	Separate data	F1	F2
[0x3000,0x4000)	Guard, no mapping	None	None	None

Each has 3 legal pages and 2 resident mappings. Four resident mappings consume only 3 unique frames because code F0 is shared. Adding per-process resident counts double-counts shared frames.

A first read 0x1004 can allocate zeroed storage and restart. Writing 0x0004 violates code permission. Reading 0x3004 is guard access, not an invitation to create a mapping merely because a free frame exists. Nonpresent PTEs need VMA/backing metadata to distinguish these outcomes.

Legality, mapping and residency (continued)

Reservation, committed backing, and residency are different quantities. A VMA can reserve addresses before policy promises RAM/swap. A virtual range larger than RAM does not guarantee all pages can be written simultaneously or that allocation can never fail.

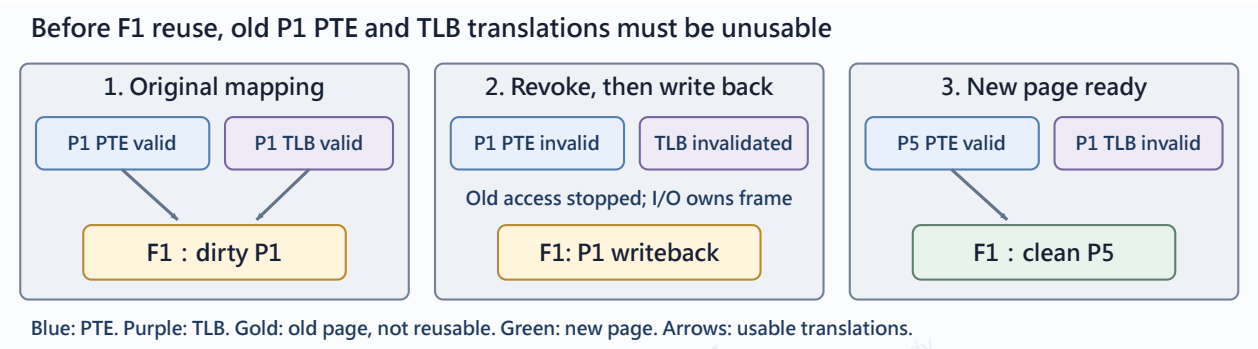
Lin Cing https://github.com/xinhi6716/computer-science-self-study

Demand-fault service and frame reuse

Fix 4 KiB pages and full frames F0→P0, F1→dirty P1, F2→P2. P5 is readable in a legal VMA/backing object but nonresident. Replacement chooses F1.

Step	State/action	Can P5 be read?
0	CPU accesses nonpresent P5 and traps	No
1	Validate VMA, backing, and read rights	No
2	Revoke P1 PTE; complete old TLB invalidation and prevent further writes to F1	No
3	Write back dirty P1; I/O still owns F1, so cannot overwrite it	No
4	Load 4096 bytes of P5 into F1; mapping not yet published	No
5	Publish P5→F1, perform required translation maintenance, restart instruction	Yes

Demand-fault service and frame reuse (continued)



This is1 fault,1 disk write,1 disk read, and one old-translation maintenance phase. On one CPU that can be local invalidation; other CPUs caching the old translation require completed shootdown. Installation may require additional architecture-specific invalidation/barriers, so this is not a claim of exactly one TLB instruction everywhere. Nonoverlapped 8 ms write plus 8 ms read costs 16 ms without other overhead. A clean victim removes the write, leaving 8 ms.

Let m be the total cost of a normal reference and F the total cost of a faulting reference through successful retry; $0 \le p \le 1$. The bound below assumes $F > m$ and $EAT_{max} \ge m$, and counts retry only once.

Demand-fault service and frame reuse (continued)

$$EAT = (1-p)m + pF, \qquad p \le \frac{EAT_{max} - m}{F - m}.$$

For m=200 ns and clean F=8,000,000 ns, EAT≤400 requires p≤200/7,999,800≈2.5001×10⁻⁵. Dirty write+read F=16,000,000 reduces it to≈1.2500×10⁻⁵. The same fault ratio can therefore have different service cost. Another fixed baseline m=100 ns, F=8 ms, EAT≤200 gives p≤100/7,999,900≈1.25×10⁻⁵; keep its denominator separate.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fault classification and stale-TLB counterexample

The handler checks legal VMA/backing, software mapping versus nonresident hardware translation, requested rights or authorized COW exception, free/victim eligibility including dirty/writeback/pin state, then complete contents/permissions before retry.

CPU0 clearing P1 PTE does not revoke CPU1's cached writable P1→F1. Loading P5 into F1 immediately lets CPU1's next P1 store corrupt P5. Reuse requires old translation revocation, necessary shootdown acknowledgments, and completed old I/O/ownership. Conversely, loading P5 without installing its mapping makes the restarted instruction fault again.

Read-ahead may bring P5/P6 in one transfer, changing fault counts and frame pressure. Every prefetched page still needs legality, backing, a frame and a consistent mapping. A batch read cannot authorize an adjacent guard page.

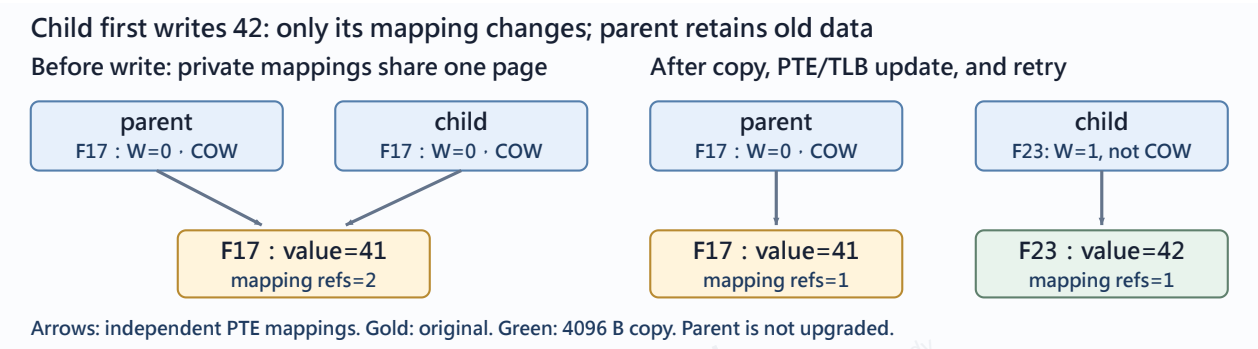
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

COW is a write-permission fault

Fork preserves separate tasks and page-table metadata while parent/child PTEs initially share F17. Hardware marks both read-only; software records COW. The translation exists, so the child's failed store is not a missing-backing-page fault.

State/event	Parent PTE	Child PTE	F17	F23
S0 fork complete	F17, W0, COW	F17, W0, COW	41, refs 2	Absent
S1 child tries write	Unchanged	Unchanged	41, refs 2	Absent
S2 allocate/copy 4096 B, not published	F17 RO	Still F17	41, refs 2	41, handler-owned
S3 publish, maintain TLB, retry 42	F17, W0, COW	F23, W1, non-COW	41, refs 1	42, refs 1, dirty

COW is a write-permission fault (continued)



Validate COW→allocate→copy→publish child PTE/TLB→retry. Making both mappings writable or clearing only the parent's COW lets the child change the parent's original data. Here refs counts user mappings, not the handler's temporary hold.

The child handler need not visit or immediately upgrade the parent's PTE. On the parent's later write fault, an exclusively owned private anonymous page with no other sharing/pin restrictions may be made writable without copying. One mapping alone does not authorize writing a shared file-cache page in place. Fork followed by exec can unmap untouched COW pages without copying. Zero-fill must clear a reused frame before exposing it to another process; demand paging, COW, and zero-fill have different fault causes and data sources.

Replacement policies and full trace

FIFO uses load order; LRU last use; OPT farthest next use, requiring an offline future; CLOCK gives referenced pages another chance. Exact LRU ordering on every access has hardware/concurrency cost. Reference-bit histories or clock scans approximate it and have their own sampling/hand rules.

Fix 3 frames, initially empty, references 1,2,3,4,1,2,5,1,2,3,4,5. H/F means hit/fault. FIFO is load order, LRU oldest→newest, CLOCK physical slots; these are policy metadata rather than one shared frame numbering.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Replacement policies and full trace (continued)

t/page	FIFO	LRU	OPT	CLOCK
1/1	[1]F	[1]F	[1]F	[1]F
2/2	[1,2]F	[1,2]F	[1,2]F	[1,2]F
3/3	[1,2,3]F	[1,2,3]F	[1,2,3]F	[1,2,3]F
4/4	[2,3,4]F	[2,3,4]F	[1,2,4]F	[4,2,3]F
5/1	[3,4,1]F	[3,4,1]F	[1,2,4]H	[4,1,3]F
6/2	[4,1,2]F	[4,1,2]F	[1,2,4]H	[4,1,2]F
7/5	[1,2,5]F	[1,2,5]F	[1,2,5]F	[5,1,2]F
8/1	[1,2,5]H	[2,5,1]H	[1,2,5]H	[5,1,2]H
9/2	[1,2,5]H	[5,1,2]H	[1,2,5]H	[5,1,2]H
10/3	[2,5,3]F	[1,2,3]F	[2,5,3]F	[5,3,2]F
11/4	[5,3,4]F	[2,3,4]F	[5,3,4]F	[5,3,4]F
12/5	[5,3,4]H	[3,4,5]F	[5,3,4]H	[5,3,4]H

Totals FIFO 9, LRU 10, OPT 7, CLOCK 9. Victims, with dash for free-frame insertions/hits:

Replacement policies and full trace (continued)

Policy	Victims
FIFO	−,−,−,1,2,3,4,−,−,1,2,−
LRU	−,−,−,1,2,3,4,−,−,5,1,2
OPT	−,−,−,3,−,−,4,−,−,1,2,−
CLOCK	−,−,−,1,2,3,4,−,−,1,2,−

FIFO hits do not reorder; LRU hits move to most recent. OPT evicts the latest next use or never-used-again page. Aging can shift the reference bit into a history byte each sampling period, with smaller values suggesting older usage. Changing sample periods, hand positions, or bit clearing changes the approximation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Trace three replacement policies (steps 1–10)

Use the same sequence `7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1` with three initially empty frames, demand loading only, and no prefetch or external invalidation. Slots are fixed physical positions; `-` is empty, H is a hit, F is a fault, and `F→x` evicts page `x`.

FIFO evicts the earliest loaded resident page; LRU evicts the least recently used page; OPT evicts the page whose next use is farthest away, with no future use treated as infinity. OPT ties choose the smallest page number.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Trace three replacement policies (steps 1–10) (continued)

t	page	FIFO : slots ; event	LRU : slots ; event	OPT : slots ; event
1	7	7,–,– ; F	7,–,– ; F	7,–,– ; F
2	0	7,0,– ; F	7,0,– ; F	7,0,– ; F
3	1	7,0,1 ; F	7,0,1 ; F	7,0,1 ; F
4	2	2,0,1 ; F→7	2,0,1 ; F→7	2,0,1 ; F→7
5	0	2,0,1 ; H	2,0,1 ; H	2,0,1 ; H
6	3	2,3,1 ; F→0	2,0,3 ; F→1	2,0,3 ; F→1
7	0	2,3,0 ; F→1	2,0,3 ; H	2,0,3 ; H
8	4	4,3,0 ; F→2	4,0,3 ; F→2	2,4,3 ; F→0
9	2	4,2,0 ; F→3	4,0,2 ; F→3	2,4,3 ; H
10	3	4,2,3 ; F→0	4,3,2 ; F→0	2,4,3 ; H

Trace three replacement policies (steps 11–20)

Continue from the previous page. Every cell gives the three fixed slots **after** the access and the event. A FIFO hit does not alter load order; an LRU hit updates recency.

t	page	FIFO : slots ; event	LRU : slots ; event	OPT : slots ; event
11	0	0,2,3 ; F→4	0,3,2 ; F→4	2,0,3 ; F→4
12	3	0,2,3 ; H	0,3,2 ; H	2,0,3 ; H
13	2	0,2,3 ; H	0,3,2 ; H	2,0,3 ; H
14	1	0,1,3 ; F→2	1,3,2 ; F→0	2,0,1 ; F→3
15	2	0,1,2 ; F→3	1,3,2 ; H	2,0,1 ; H
16	0	0,1,2 ; H	1,0,2 ; F→3	2,0,1 ; H
17	1	0,1,2 ; H	1,0,2 ; H	2,0,1 ; H
18	7	7,1,2 ; F→0	1,0,7 ; F→2	7,0,1 ; F→2
19	0	7,0,2 ; F→1	1,0,7 ; H	7,0,1 ; H
20	1	7,0,1 ; F→2	1,0,7 ; H	7,0,1 ; H

Across 20 accesses FIFO incurs **15**, LRU **12**, and OPT **9** faults. At step 8, LRU compares pages 2, 0 and 3 last used at steps 4, 7 and 6, so it evicts 2. OPT sees 0 next used at step 11, later than 2 at step 9 and 3 at step 10, so it evicts 0.

Every CLOCK hand and reference bit

Start hand 0. Fill free slots 0/1/2 with R1 without moving hand. Hits set the page's R1 without moving hand. When full, clear each encountered R1 and advance; replace first R0, set new R1 and advance once.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Every CLOCK hand and reference bit (continued)

t/page	Slots 0/1/2	R0/R1/R2	Next hand	H/F, victim
1/1	[1]	[1]	0	F,−
2/2	[1,2]	[1,1]	0	F,−
3/3	[1,2,3]	[1,1,1]	0	F,−
4/4	[4,2,3]	[1,0,0]	1	F,1
5/1	[4,1,3]	[1,1,0]	2	F,2
6/2	[4,1,2]	[1,1,1]	0	F,3
7/5	[5,1,2]	[1,0,0]	1	F,4
8/1	[5,1,2]	[1,1,0]	1	H,−
9/2	[5,1,2]	[1,1,1]	1	H,−
10/3	[5,3,2]	[0,1,0]	2	F,1
11/4	[5,3,4]	[0,1,1]	0	F,2
12/5	[5,3,4]	[1,1,1]	0	H,−

At4 it clears a full circle then evicts 1 in slot 0; at 7 another circle evicts 4. The initialization and hit behavior are necessary to reconstruct these states.

FIFO anomaly with both capacities

t/page	3-frame queue	4-frame queue	Cumulative faults 3/4
1/1	[1]F	[1]F	1/1
2/2	[1,2]F	[1,2]F	2/2
3/3	[1,2,3]F	[1,2,3]F	3/3
4/4	[2,3,4]F	[1,2,3,4]F	4/4
5/1	[3,4,1]F	[1,2,3,4]H	5/4
6/2	[4,1,2]F	[1,2,3,4]H	6/4
7/5	[1,2,5]F	[2,3,4,5]F	7/5
8/1	[1,2,5]H	[3,4,5,1]F	7/6
9/2	[1,2,5]H	[4,5,1,2]F	7/7
10/3	[2,5,3]F	[5,1,2,3]F	8/8
11/4	[5,3,4]F	[1,2,3,4]F	9/9
12/5	[5,3,4]H	[2,3,4,5]F	9/10

FIFO anomaly with both capacities (continued)

After 7 the smaller set contains 1 but the larger does not; access 8 faults only with 4 frames. FIFO's capacity-dependent fault history determines load order, so increasing capacity can raise faults. This Belady anomaly does not apply to every policy.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

General LRU inclusion proof

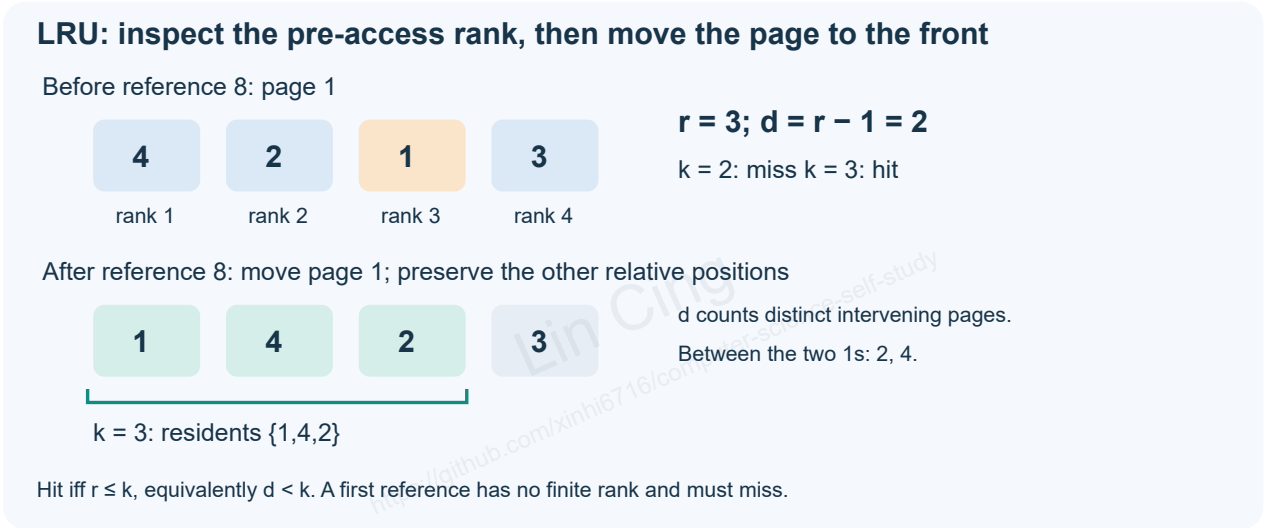
Form one recency stack of every seen page from newest to oldest, independent of capacity. On every access move that page to the front. Inductively, exact k-frame LRU holds the first k entries, or all entries when fewer exist. Initially empty; a hit reorders without changing membership of those k; a miss inserts at front and pushes out old entry k, exactly the LRU victim.

Thus $R_k(t) \subseteq R_{k+1}(t)$. If k+1 faults at time t, its requested page was absent from R(k+1) before access and therefore absent from subset Rk. Fault indicators satisfy the inequality pointwise, and summing gives $F_{k+1} \leq F_k$. The finite 12-access example cannot replace this induction. FIFO load histories and CLOCK hand/reference scans depend on capacity and do not supply that same stack invariant.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Predict LRU hits with stack distance

Start empty with equal-sized pages and demand paging; exclude prefetching, pins, and external invalidations. Rank r is the 1-based position before access, newest first.



Reuse distance d counts distinct intervening pages: d=r−1. Some texts call r the stack distance; check whether indexing starts at 0 or 1.

One trace predicts faults at two capacities

t	Page	Pre-access r	Post-access order: newest → oldest	LRU 2	LRU 3
1	1	First	1	F	F
2	2	First	2,1	F	F
3	1	2	1,2	H	H
4	3	First	3,1,2	F	F
5	1	2	1,3,2	H	H
6	2	3	2,1,3	F	H
7	4	First	4,2,1,3	F	F
8	1	3	1,4,2,3	F	H

Faults: 6 with two frames; 4 with three. F denotes a fault and H a hit. Before reference 8, page 1 has rank 3: two-frame LRU has evicted it, whereas three-frame LRU retains it.

Why is the hit condition d < k?

Immediately after using page p, it is first. Each distinct other page subsequently referenced places one page ahead of p; repeating a page already ahead does not increase that number. Thus r=d+1 before the next use. Exact k-frame LRU retains precisely the first k pages, so a hit is equivalent to $r \leq k$, or $d<k$. First references must fault.

A working-set window counts references; an LRU capacity counts distinct pages. At t=7 the four-reference window contains four pages. Three-frame LRU holds only {4,2,1}; page 3 remains in the working set but has been evicted. WS(t,4) is not automatically the resident set of four-frame LRU.

OPT exchange proof and its cost boundary

Fix an offline finite string, identical initial residents, unit miss cost, and no dirty cost difference. At a full-frame miss OPT evicts q with latest next use, treating never again as infinity. Take an optimal schedule's first disagreement, evicting p instead. Modify it to retain p and evict q. Before next(p), q is not requested earlier, so other replacements can track the original with only p/q differing. If the original evicts q sooner the states can align. Otherwise at next(p), the modified schedule already has p while the original faults. Use that saved transfer to bring q in and remove the original's chosen victim, aligning states without more transfers. If neither page is reused, the difference costs nothing.

Any early transfer in that alignment can be delayed until the page's first request without increasing cost, giving a legal demand schedule. Hence an optimal schedule agrees with this OPT step; eliminate first disagreements successively. This is a unit page-in proof, not a proof for heterogeneous page sizes, dirty page-out, or unequal I/O weights. More frames can simulate any smaller-capacity optimum by leaving space unused, so OPT's minimal cost cannot rise. This simulation does not require every OPT tie-break to produce nested residents.

Fault service is trap+dirty swapout+swapin. Half dirty victims gives an expectation between read-only and write+read paths. Minimum fault count need not minimize a dirty-weighted cost.

LFU/MFU frequency lifetimes

Start 3 empty frames, count 1 on load, increment on hit, discard count on eviction, and break ties by earliest load. References 1,1,1,2,3,4 give counts{1:3,2:1,3:1} before the final access. LFU evicts 2→{1:3,3:1,4:1}; MFU evicts 1→{2:1,3:1,4:1}. Old high counts can preserve 1 after locality shifts to 2/3/4. Aging counts can reduce stale history but adds a decay parameter. MFU's belief that low counts are newly loaded future candidates is a workload assumption, not universal optimality.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Quotas, rounding and victim pools

Equal allocation first gives floor(F/m) each and needs a remainder rule. Proportional allocation floors $a_i = Fs_i / \sum s_j$, then distributes remaining frames by specified remainder/priority/order.

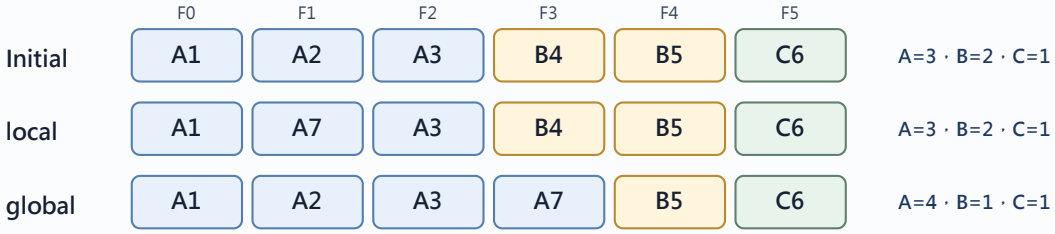
For F64 and sizes A100/B200/C300, raw shares 10.667/21.333/32, floors 10/21/32 total 63. Largest remainders 0.667/0.333/0 give the last frame to A, final 11/21/32 total 64.

In a separate six-frame model, A={1,2,3}, B={4,5}, C={6}; last-use A1=10, A2=5, A3=8, B4=2, B5=7, C6=9. A faults 7 with no free frame.

Policy	Eligible victim	Victim	New residents
Local	Only A	A2	A={1,7,3}, B={4,5}, C={6}
Global	All processes	B4	A={1,2,3,7}, B={5}, C={6}

Quotas, rounding and victim pools (continued)

One A7 fault: local/global each reuses one frame



All rows have six frames. After global eviction, the next B4 access faults.

Global raises A's residency 3→4, lowers B2→1, and retains 6 total. It neither deletes A3 nor invents B8. B's next access 4 now faults, spreading pressure. Local limits interference but cannot borrow an idle other's frame. Allocation defines quotas/bounds; replacement defines eligible victims. Global need not mean unrestricted admission.

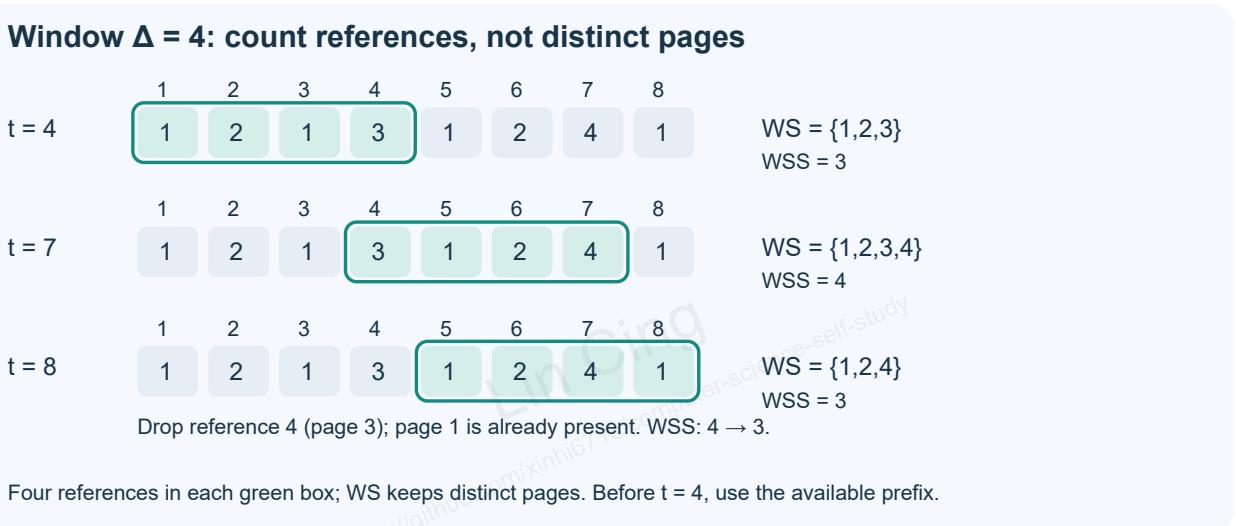
Minimum frames versus a working set

Instruction code/source/destination pages and restart rules create a minimum progress requirement. Evicting an earlier required operand to load the last can fault forever when an instruction cannot partially complete. That is different from WSS, a performance-demand estimate over a chosen window. Fault-handler and table kernel memory are separately guaranteed system costs.

If lower bounds li apply, require Σli≤F. Grant them first, then split F−Σli with explicit rounding. Taking max(proportional_share, li) independently can exceed F. For F6 and three processes each requiring 3 unshared frames, total minimum 9 is impossible regardless of fairness. Suspend at least one, change execution, or add frames. Pins and kernel reserves also reduce the user pool.

A moving working set is not a fixed-capacity cache

Let p_t be the page at reference t, and choose $\Delta = 4$.



$WS(t,4) = \{p_j : \max(1, t-3) \le j \le t\}$, with $WSS = |WS|$. At t=8 the window [1,2,4,1] contains three distinct pages, not four.

An expired reference need not remove its page

Maintain a count for each page: increment for the incoming reference and decrement for the outgoing one. Remove the page from WS only when its count becomes zero.

t	Incoming	Outgoing reference	Window	Working set	WSS
4	3	None	1,2,1,3	{1,2,3}	3
5	1	Reference 1: page 1	2,1,3,1	{1,2,3}	3
6	2	Reference 2: page 2	1,3,1,2	{1,2,3}	3
7	4	Reference 3: page 1	3,1,2,4	{1,2,3,4}	4
8	1	Reference 4: page 3	1,2,4,1	{1,2,4}	3

Do not remove page 1 at t=5: only one reference expires, and two references to page 1 remain. At t=8 the final reference to page 3 expires, so WSS decreases.

Working sets and evidence of thrashing

Thrashing means paging dominates useful execution. With a four-reference window:

Process	References	Working set	WSS
A	1,2,3,1,2,4,5,1	{1,2,4,5}	4
B	6,7,6,8,7	{6,7,8}	3
C	9,9,9	{9}	1

Without sharing, demand D=8 exceeds six available frames. Active localities cannot all remain resident. For shared pages, count unique backing pages rather than summing WSS.

Excess demand predicts pressure; it does not measure thrashing. A short window misses reused pages; a long one retains stale demand. Check fault rates, useful CPU utilization, and paging queues. A lock or one slow I/O can also leave the CPU idle.

PFF hysteresis controller

For fixed 5-access windows and upper 0.4/lower 0.1, FFFFF gives 1.0 and requests another frame; FHHHH gives 0.2 and holds; HHHHH gives 0 and may release a spare under pressure. Two thresholds avoid allocation/reclaim oscillation near one threshold.

```
for each process every sample_window:
    if accesses_in_window == 0: continue
    rate = faults_in_window / accesses_in_window
    if rate > upper and free_frames > 0:
        grant_one_frame(process)
    else if rate > upper and free_frames == 0:
        choose one process to suspend
    else if rate < lower and memory_pressure and quota > minimum_required:
        reclaim_one_spare_frame(process)
```

High PFF with no free frame calls for reduced multiprogramming, swapping/reallocation, or rejected admission. More CPU priority cannot manufacture resident pages. Short windows are noisy, long windows react slowly; thresholds, window and suspension rule jointly define the controller.

Mapped files, visibility and durability

MAP_SHARED processes map the same file-cache page; shared updates still require application synchronization. MAP_PRIVATE writes COW into private pages without writing the original file. Creating a mapping is not residency; a store is not persistence.

These experiments independently start at 7, with no external file modification.

Shared event	P1	P2	Cache F40	File
S0	F40	F40	7, clean	7
S1 P1 writes 8 and synchronizes	F40	F40	8, dirty	7
S2 P2 reads after synchronization	F40	F40	Reads 8, not durable	7
S3 completed required flush	F40	F40	8, clean	8

Private event	P1/value	P2/value	Cache	File
V0	F40/7, COW	F40/7, COW	F40/7	7
V1 only P2 writes 9	F40/7, uncopied	New F53/9	F40/7	7
V2 P1 writes 8	New F52/8	F53/9	F40/7	7

V1 copies 4096 B; cumulative V2 copies 8192 B. P2's write does not copy the still-unwritten P1 mapping. MAP_PRIVATE does not guarantee whether later external file changes appear, so this is not an automatic file snapshot.

Mapped files, visibility and durability (continued)

msync/platform counterparts trigger writeback/order under their actual API/filesystem/cache/flush contracts. Dirty shared file pages need safe writeback before reclaim. Dirty private COW pages use swap/backing rather than overwriting the source file. Failed writeback must retain error/dirty state instead of declaring clean and reusing data.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

mmap and read/write costs

Fix cold cache, one complete 4 KiB page, no read-ahead, successful operations. Disk→cache I/O is different from cache↔user CPU copy.

Path	Calls	CPU copies	Translation	Write durability
read	1 syscall	Cache→user 4096 B	Ordinary	No write promise
write	1 syscall	User→cache 4096 B	Ordinary	Requires specified sync/commit
First mapped read	Earlier mmap call; current fault/trap	No cache→user copy; disk still reads 4096 B	Install/maintenance	No write
Mapped shared store	No syscall until flush	Zero bytes in this model	Access/dirty tracking	Specified msync/flush needed

read updates file offset; mapped loads use MMU after faults create PTEs. A page fault is an exception, not an explicit user syscall. Both write and mapped store can finish with only dirty cache data.

Kernel allocation and buddy splitting

Kernel interrupt/I/O paths may need nonsleeping, contiguous, or specially owned memory, so ordinary user demand-paging assumptions do not suffice. Buddy allocators split power-of-two blocks; slabs cache fixed-size objects.

A256 KiB region with 4 KiB minimum rounds 21 KiB up to 32. Ranges below use KiB.

Step	Split/result
0	Free[0,256)
1	[0,128),[128,256)
2	Split left into[0,64),[64,128)
3	Split left into[0,32),[32,64)
4	Allocate[0,32),21 used and 11 internal waste

Free blocks[32,64),[64,128),[128,256) total 224 KiB. A64 KiB request can use an existing 64 block; coalescing[0,64) specifically requires releasing its allocated 32 first. Blocks have unique order/buddy ownership and never overlap. For appropriately aligned a and block size 2^k bytes, buddy=a xor 2^k. Freeing[0,32) merges with[32,64), then[64,128), then[128,256). Each step checks only the matching-order buddy; maximum tree height log2(256/4)=6.

Slab object counts and the denominator

Ignoring metadata, a4096 B slab page holds 16 objects of256 B. This is a model, not an exact count for every slab implementation. Allocate 2 and free 1: live objects 1, free objects 15, live bytes 256, retained slab bytes 4096. Process RSS cannot be inferred from kernel object count because it uses process mappings as its denominator.

Retaining free objects saves initialization and page-allocation work, so footprint need not shrink with live count. NUMA caches add node locality and remote-reclaim/allocation cost.

Page size, prepaging and pins

One MiB needs 256 PTEs with 4 KiB pages or16 with 64 KiB pages. Larger pages give 16 times coverage per TLB entry but larger tails, transfer sizes, and read amplification. The number of faults does not decrease monotonically with page size; page-table cost, TLB coverage, transfer size and tail waste require separate comparisons.

An8-page workload uses locality{P1, P2, P3}. Prepaging those 3 does 3 reads and the later first 3 accesses fault 0 times. Pure demand does the same 3 reads but 3 faults. The difference is trap/latency placement, not eliminated data I/O. Guessing unused P4 adds a needless read.

DMA stage	F40 pin/state	Reclaimable?
t0 user buffer	Pin 0	Yes if otherwise eligible
t1 device queued	Pin 1, DMA pending	No
t2 device completes	DMA 0, pin not released	No
t3 completion releases pin	Pin 0, eligible LRU	Can be considered

Reusing at t1 lets DMA overwrite a different page. Device completion alone does not end the completion protocol; pins need normal/error release paths. A pin, lock bit and mlock are different API mechanisms, but none allows duplicate allocation of a still-used frame. Long pins shrink the evictable pool.

Page size, prepaging and pins (continued)

Counterexample: larger pages need not reduce faults. Fix a 256 KiB frame budget for this workload, initially empty, exact LRU, demand paging, no prefetch and no competing references. Cycle through five distinct 64 KiB regions, touching the same single 4 KiB subpage in each. With 4 KiB pages there are 64 frames: five initial faults then hits. With 64 KiB pages there are only four frames: cyclic LRU faults on every access. Fix both the byte budget and the reference pattern before comparing.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

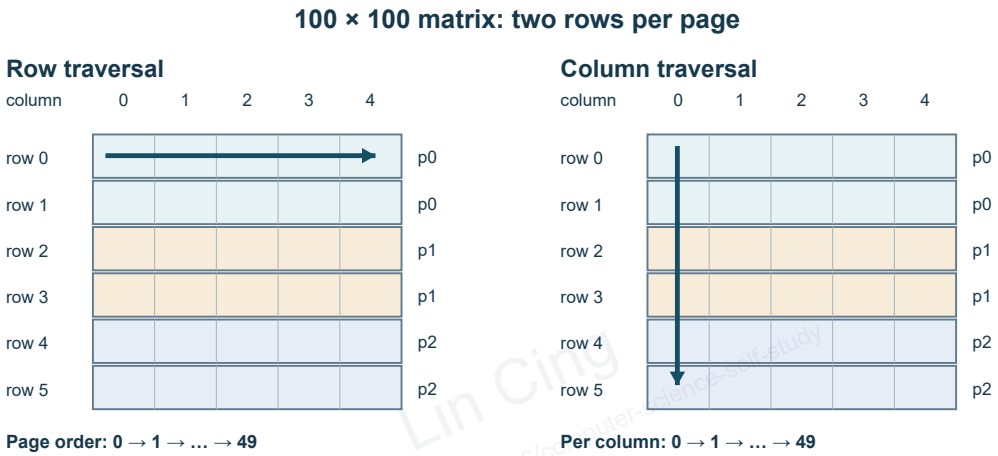
Access order changes page locality

C/C++ row-major int A[4][1024],4-byte int,4 KiB-aligned base,4 KiB pages makes one row per page. Count only array references,2 initially empty exact-LRU frames; code/stack/cache-line costs are separate.

Loop order	Page references	Faults
Rows outside, columns inside	Each 0,1,2,3 repeated 1024 times	4, first access per row
Columns outside, rows inside	0,1,2,3 repeated 1024 rounds	4096, each reuse already evicted

First 8 column states oldest→newest are[0]F,[0,1]F,[1,2]F,[2,3]F,[3,0]F,[0,1]F,[1,2]F,[2,3]F, then repeat. Identical 16 KiB and operation count produce different fault counts under this residency model. Real OS frame grants/prefetch/policies can change that difference; it is not a measured universal loop speed ratio.

Matrix traversal changes the page sequence



First 6 rows and 5 columns shown; each pair of rows shares a page.

The matrix has 100×100 one-byte elements in row-major storage, starting at a 200-byte page boundary. Each page holds exactly two rows, so there are 50 pages. Count only matrix data; code and other data do not compete for the data frames.

Pages are numbered from 0. Row i (i=0…99) is on page `floor(i/2)`. Each traversal starts with an empty frame set.

Why does column traversal incur 5,000 faults?

Traversal	Data frames, policy	Page order to check	Faults
Row by row	2, LRU	Use each page for 200 consecutive accesses before the next	50
Column by column	2, LRU	Each column visits pages 0,1,…,49; two adjacent accesses per page	100×50 = 5,000
Column by column	50, LRU	First column loads all pages; later accesses hit	50

Before page 0 is reused in the next column, the traversal touches 49 other distinct pages; two frames cannot retain it. Within each column the two rows of one page are adjacent: the first access faults and the second hits. With 50 frames the whole matrix remains resident. Page size, alignment, frame capacity and policy all affect the result; traversal order alone does not determine the fault count.

Linux 3.2 reclaim branches

Fix asynchronous order-0 Linux v3.2. Free pages below the zone low watermark can wake kswapd; scanning does not guarantee every candidate frees memory. It scans inactive anon/file lists and balances active/inactive. page_check_references distinguishes PTE references and PageReferenced; shrink_page_list then tests unmap, dirty, writeback, mapping/refcounts. This is not modern multi-gen LRU.

Fix current_is_kswapd=true, priority 9, DEF_PRIORITY12, may_writepage=true, filesystem/I/O/unmap allowed, successful page locks, no concurrent rereference/mutation, removable P2/P3 mapping/refcounts, successful asynchronous P3 pageout, non-lumpy reclaim for P1, and PageReferenced=0 for all five candidates. The PTE-referenced column is a distinct state. P4 is an already-isolated candidate that became mlocked before its unevictable cull completed; normally an mlocked page is not a stable ordinary inactive-LRU candidate.

Linux 3.2 reclaim branches (continued)

Page	Type	PTE referenced	Dirty	Writeback	mlocked	Action	Free delta
P1	Anon	1	0	0	0	Activate	0
P2	File	0	0	0	0	Reclaim after final removal	+1
P3	File	0	1	0	0	Start async writeback	0
P4	Anon	0	0	0	1	Cull to unevictable	0
P5	File	0	1	1	0	Defer	0

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Linux 3.2 reclaim branches (continued)

Linux 3.2: outcomes for five candidate pages

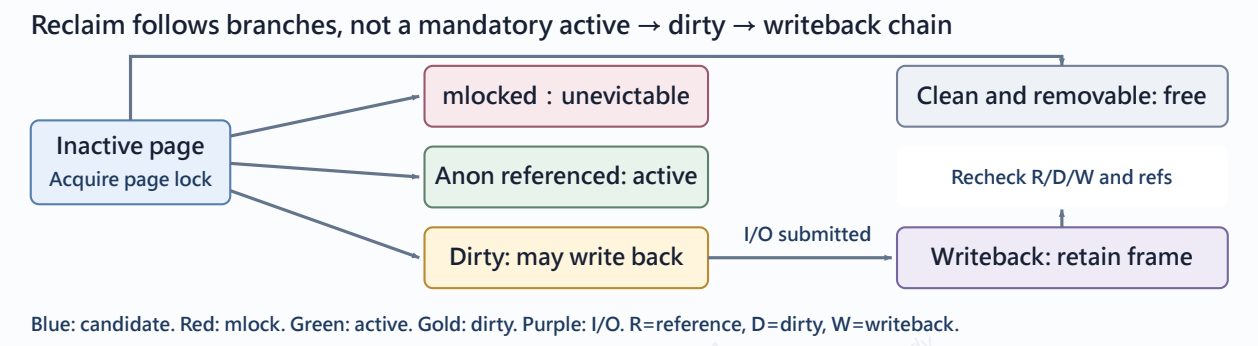
Teaching snapshot: order-0, kswapd, asynchronous writeback; P4 isolated before culling

Page	Conditions driving the decision		Action in this scan	Freed
P1	Anonymous; PTE referenced=1	→	Activate to active LRU	0
P2	File; unreferenced, clean; removable	→	Reclaim and free frame	+1
P3	File; dirty; PageReferenced=0	→	Submit writeback; retain	0
P4	mlocked; isolated before culling	→	Move to unevictable	0
P5	PageWriteback=1	→	Defer	0

P3 pageout starts successfully. After writeback, recheck dirty and references before reuse.

Only P2 frees a frame now. P3 is a selected candidate but remains owned by I/O. An extra DMA pin/reference is not mlock; it may remain on ordinary LRU and fail final mapping/reference removal rather than become unevictable.

Recheck after writeback and changed conditions



Writeback completion does not guarantee no subsequent dirty write. Before reuse recheck dirty 0, writeback 0, removable mappings/references. Mlocked pages require authorized unlock; time alone cannot clear ownership. Free count increases only after actual reclaim.

Recheck after writeback and changed conditions (continued)

Changed condition	v3.2 branch/result
P1 instead file, first PTE reference, PageReferenced 0	Set PageReferenced, PAGEREF_KEEP; remain inactive rather than unconditional activate
P3 priority 10	priority≥DEF_PRIORITY−2; retain/mark reclaim, other writeback path
P3 direct reclaim, not kswapd	Dirty file-cache restriction blocks this pageout
P3 PageReferenced=1, PTE referenced=0	Non-swap-backed file page yields PAGEREF_RECLAIM_CLEAN; a still-dirty page is kept, not sent to the original pageout branch
P3 may_writepage=false	shrink_inactive_list may use ISOLATE_CLEAN and exclude dirty pages before isolation; an already isolated page reaching shrink_page_list is kept by its may_writepage check. In either case this path does not page out P3 or free it
P2 extra pin/reference	Final removal fails, retain; not synonymous with unevictable
P5 completed I/O then redirtied	Dirty branch again; no clean reuse

Recheck after writeback and changed conditions (continued)

A single if-referenced-activate/if-dirty-writeback sketch omits page type, reference source, scan context, and final ownership. Windows working-set limits, Solaris lotsfree/desfree/minfree, and Linux watermarks are distinct version-specific controls.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Historical Windows trimming

Use the OSC 9e simplified Windows working-set model, cross-read with Chapter 19's Windows 7 snapshot. It is not a guarantee for every current patch. Min/max control residency; low free memory can trigger trimming above minima.

Set threshold 3, free 1; A(min 2, resident 4, max 4), B(2,3,4), C(2,2,4). All selected pages are clean/unlocked; visit A→B→C and stop only once free>3.

Action	A	B	C	Free
Initial	4	3	2	1
Trim 2 from A	2	3	2	3
Trim 1 from B	2	2	2	4
Do not trim C	2	2	2	4

A later fault can grow its set below maximum if free pages exist; at maximum this model uses local replacement. Clustered paging can add neighbors, so exactly one resident page per fault belongs only to the earlier single-page model.

Historical Solaris two-hand window

In the OSC 9e snapshot free<lotsfree starts the scanner, whose scanrate increases from slowscan under pressure. desfree/minfree prompt stronger reclaim/swap measures. These are not Linux's same low watermark.

Front clears R; back revisits after h/s seconds for hand distance h pages and steady rate s pages/s. h4, s100 gives 0.04 s, illustrative rather than a default.

Page	After front clears R	Back observes	Action
Q1	Read again, R1	R1	Retain locality
Q2	Unused, clean	R0, clean	Unmap, then reclaim/cache list
Q3	Unused, dirty	R0, dirty	Write back; cannot reuse during I/O

Raising s to 400 with h4 shortens the window to 0.01 s, making absence of a recent reference more likely. An unoverwritten clean cache-list page may be recovered; invalid free-list contents are not a retained-data guarantee.

Complete access decision and cost model

Stage	Required evidence	Invalid inference
Range/backing	Authorized mapping and content source	Every nonpresent fault authorizes page-in
Rights/COW	Requested operation allowed or valid COW	Every forbidden write gets a new frame
Residency/PTE	Frame, standby, or storage requirement	Every fault means one disk I/O
Victim/writeback	Unmap eligibility and safe dirty handling	Pinned/dirty frames can be reused immediately
Restart	Contents, mapping, rights coherent	This page can never refault

Capacity counts unique frames; COW/unmap/replacement must track final mapping, TLB, and I/O users. With the fixed 12 references,3 frames,100 ns hits, and 8 ms total fault service including that access:

Complete access decision and cost model (continued)

Policy	Faults	Hits	Total cost
FIFO	9	3	72 ms+300 ns
LRU	10	2	80 ms+200 ns
OPT	7	5	56 ms+500 ns
CLOCK	9	3	72 ms+300 ns

OPT uses future information for a lower bound. Different fault costs, prefetch, or clustering invalidate this table's common multiplier. Working sets, quotas, PFF, and scanning jointly affect residency; evaluate useful throughput and waits as well as fault counts.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 9. Linux v3.2 vmscan.c; Linux v3.2 memory.c; mmap(2).

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study